

BACKEND ENGINEERING ASSIGNMENT

Tokenized Real Estate Search & Ingestion Engine Assessment

1. Project Background

You are building the core **Search, Suggestion, and Asset Ingestion Engine** for a high-performance tokenized real estate platform. On this platform, real estate assets are fractionalized into tradeable digital tokens, allowing users to interact with and buy into two distinct financial categories:

1. **New Launches:** Primary issuance of asset tokens (conceptually identical to a Mutual Fund NFO).
2. **Marketplace:** Secondary peer-to-peer trading ecosystem of existing property fractions.

Because our core matching engine operates on ultra-fast order matching algorithms bound strictly to `Int64` (`BigInt`) Primary Keys, our search infrastructure must be a specialized, hyper-lean, read-optimized data layout that maps natural language queries to structured `Int64` references instantly.

2. Core Architectural Mandate (The Tech Stack)

To optimize infrastructure expenditure, minimize operational maintenance, and maximize raw performance for an ecosystem scaling up to **1,000,000 users over 5 years**, you will avoid heavy enterprise distributed clusters (like standard PostgreSQL installations or complex Elasticsearch multi-node configurations) for this version. Instead, you are mandated to employ a modern, fast, file-and-memory footprint stack:

- **Backend Framework: Rust** (utilizing a robust asynchronous runtime ecosystem like `axum` or `actix-web`).
- **Source of Truth Database: SQLite** (leveraging `sqlx` or `rusqlite` explicitly configured for production-level multi-threaded concurrency).
- **Speed & Fuzzy Search Index: Meilisearch** (The Rust-native, low-latency, full-text search engine optimized for search-as-you-type suggestions).
- **Frontend Client Demo: Flutter** (Web-only distribution target, focusing on a highly fluid UI interaction loop).

3. Assignment Scope & Requirements

Part A: Secured Data Ingestion API (Rust)

Before asset metadata can become discoverable, it must be ingested into our tracking layers. You must construct a secure administrative endpoint to facilitate inventory onboarding:

- **Endpoint Path:** `POST /api/v1/admin/properties`
- **Access Control & Security:** Protect this endpoint utilizing an authentication middleware layer (e.g., Bearer Token validation or a custom API Key validator).

- **IP Isolation Strategy:** For this local evaluation phase, the code must implement a standard access structure restricting unauthorized entry while ensuring your architecture is ready to dynamically resolve an environment-configured IP whitelist array in production.
- **Pipeline Execution Flow:** Upon receiving a valid payload, the API handler must execute a local transaction writing the comprehensive relational schema records safely into the SQLite instance. Once committed, the handler must immediately execute a non-blocking asynchronous task (e.g., utilizing `tokio::spawn`) to flatten, denormalize, and upsert the matching structure directly into the Meilisearch index cluster.

Part B: Location-Aware Search & Suggestion Engine (Rust)

Implement a single public, high-speed HTTP lookup routing endpoint intended to back the user-facing autocomplete feature:

- **Endpoint Path:** `GET /api/v1/search?q={query}&country={iso_country_code}`
- **Typo Tolerance Requirement:** The system must natively and smoothly map misspelled search sequences to real records (e.g., matching "Mubai" to "Mumbai", or "Mrina" to "Marina").
- **IP-Context Boosting Matrix:** The engine must leverage the incoming 2-character ISO country code parameter to enforce context relevancy. If a user queries the term "Marina" from an IP matching the UAE (`country=AE`), the engine must dynamically prioritize and rank "Dubai Marina" entries over other global options. If a user queries from Singapore (`country=SG`), it must prioritize "Marina Bay".
- **The Strict "Zero Leaks" Rule:** If a user inputs an explicit geographical or title pattern that registers zero true or fuzzy hits within our defined index parameters, the system must strictly return an empty array structure (`[]`). No loosely correlated fallback garbage is permitted to saturate the autocomplete UI.

Part C: Multi-Tier Advanced Filtering Strategy

To keep the search discovery subsystem incredibly lean and highly responsive, filtering operations are deliberately bifurcated across the two core datastores based on query intent:

1. **Search Discovery Layer (Meilisearch):** This system handles high-level structural parameters that fundamentally isolate broad user categories. These must be indexed as filterable attributes within the Meilisearch cluster:
 - `listing_type` (Strictly bounded to: `NewLaunch` or `Marketplace`)
 - `status` (Strictly bounded to: `Active` or `SoldOut`)
2. **Exhaustive Data Layer (SQLite):** This layer stores the descriptive relational parameters. When a user executes highly precise granular filters, the system must filter directly inside SQLite using the `Int64` ID values isolated during the discovery stage:
 - `property_type` (Enums: `Office` , `Warehouse` , `OtherCommercial` , `Farmhouse` , `Residential` , `Shop`)
 - `total_property_value` (The absolute valuation mapping of the underlying physical real estate asset)
 - `token_size` (The calculated capital required to acquire a single fractional token share)

Part D: Flutter Proof-of-Concept Interactive Frontend

Build a minimal, clean, web-targeted Flutter interface that showcases the system response capabilities:

- **The Search Component:** A centralized input bar executing an immediate backend polling routine upon user keystroke updates. It must include appropriate client-side debounce logic to avoid overwhelming the network layout.
- **The Dynamic Suggestion Tray:** Render a drop-down overlay grouping suggestions explicitly by their classified type taxonomy (e.g., grouping *Property Matches* separate from broad *Locality* or *City Matches*) exactly like top-tier aggregate property platforms.
- **The Filterable Results Interface:** Pressing "Enter" or selecting an auto-complete suggestion element must move the user to a standard list view. This view should display corresponding properties alongside functional UI control toggles allowing the application of the downstream exhaustive filter set (Type, Value, Token Cost).

4. Technical Schema Definitions

Your implementation must explicitly map to and honor the following technical definitions:

SQLite Relational Schema

```
CREATE TABLE geo_entities (  
  id BIGINT PRIMARY KEY,           -- Int64 Global ID (e.g., OpenStreetMap ID format)  
  name TEXT NOT NULL,             -- e.g., "Mumbai", "Bandra West", "Dubai Marina"  
  entity_type TEXT NOT NULL,      -- 'City', 'Locality', 'State', 'Country'  
  parent_id BIGINT,              -- Links Localities to Cities (Self-referencing)  
  country_iso TEXT NOT NULL,      -- 2-character ISO code (e.g., "IN", "AE", "SG")  
  latitude REAL NOT NULL,         -- For future spatial-mapping compatibility  
  longitude REAL NOT NULL  
);  
  
CREATE TABLE properties (  
  id BIGINT PRIMARY KEY,           -- Int64 ID synchronized with the Matching Engine  
  display_name TEXT NOT NULL,      -- Public title of the listing  
  geo_entity_id BIGINT NOT NULL,   -- Foreign Key pointing to geo_entities  
  listing_type TEXT NOT NULL,      -- 'NewLaunch' or 'Marketplace'  
  property_type TEXT NOT NULL,     -- 'Office', 'Warehouse', 'OtherCommercial', etc.  
  status TEXT NOT NULL,            -- 'Active', 'SoldOut'  
  total_property_value NUMERIC,    -- Financial valuation field  
  token_size NUMERIC,              -- Cost per individual fractional unit  
  FOREIGN KEY(geo_entity_id) REFERENCES geo_entities(id)  
);
```

Meilisearch Optimized Flattened Document Structure

```
{  
  "id": "prop_9211",  
  "target_id": 9211,  
  "display_name": "Godrej Horizon",  
  "location_text": "Dadar, Mumbai, Maharashtra, India",  
  "country_iso": "IN",  
  "entity_type": "Property",  
  "listing_type": "Marketplace",  
  "status": "active",  
  "_geo": {  
    "lat": 19.0176,  
    "lng": 72.8472  
  }  
}
```

Verified API JSON Response Format (Polymorphic Pattern)

When the Flutter client triggers a lookup via `GET /api/v1/search`, the Rust engine must yield a flat list matching this explicit formatting structural pattern:

```
[
  {
    "id": "loc_50",
    "target_id": 50,
    "display_name": "Dubai Marina",
    "secondary_text": "Dubai, UAE",
    "entity_type": "Locality",
    "listing_type": null
  },
  {
    "id": "prop_9211",
    "target_id": 9211,
    "display_name": "Marina Heights Tower",
    "secondary_text": "Dubai Marina, UAE",
    "entity_type": "Property",
    "listing_type": "Marketplace"
  }
]
```

Validation Note for Candidates: If the `entity_type` returned represents an aggregate geographic grouping like a `Locality` or `City`, the corresponding `listing_type` key **must be explicitly null**. This signals to the Flutter client that the user is selecting a regional destination bounds rather than an isolated structural property asset.

5. Mandatory Seed Data Constraints

To successfully pass validation checks and enable robust testing of your location-aware ranking rules, the submission database seeding engine must automatically deploy realistic geo-structures. The mock database must include:

- **Domestic Operations (India):** Full hierarchical mappings containing overlapping locations across **Mumbai, Pune, Indore, and Delhi**. Include highly recognizable landmark nodes (e.g., *Connaught Place* in Delhi, *Bandra* in Mumbai, and *Palasia* in Indore) linked correctly via parent keys.
- **International Identity Collision (The "Marina" Test Vector):**
 - Insert "Dubai Marina" bound to the country code `AE`.
 - Insert "Marina Bay" bound to the country code `SG`.
 - Insert a property asset containing the string token "Marina" inside its title string located inside India (`IN`).

6. Submission Guidelines & Evaluation Rubric

Technical Deliverables

1. **Mono-Repository Structure:** Deliver a clean, well-factored workspace layout tracking the backend Cargo project parallel to your Flutter deployment directory.
2. **Automated Initialization Scripts:** Include an immediate execution bootstrap mechanism that sets up the database file, applies WAL mode configurations, mounts Meilisearch ranking indexes, and injects the required Indian and international asset datasets.
3. **Architecture Analysis Document (`ARCH.md`):** Provide a brief document detailing your SQLite connection pooling strategy under concurrent Tokio load states, your exact Meilisearch ranking rule override configuration matrix, and local execution benchmark profiling readouts.

Evaluation Criteria

Component	Target Standard Expected
SQLite Concurrency Tuning	Proactive enforcement of <code>PRAGMA journal_mode=WAL;</code> and <code>PRAGMA synchronous=NORMAL;</code> to safely maximize throughput.
Idiomatic Rust Integrity	Zero panics, zero unhandled <code>unwrap()</code> invocations, unified error encapsulation via <code>thiserror</code> or <code>anyhow</code> , and proper async background task handling.
Interface Fluidity	Excellent client-side keystroke debouncing preventing UI lag or network queuing bottlenecks.
Latency Profile	Total local end-to-end network execution for search queries must register consistently under a <15ms threshold.